

**UDF - CENTRO UNIVERSITÁRIO DO DISTRITO FEDERAL**

**Ciências da computação.**

**DISCIPLINA – ITNERFACE E JORNADA DO USUÁRIO**

**NOME DO SISTEMA: NEXUS**

**Alunos:**

**Gabriel Martins Sales – 43663435**

Link do protótipo interativo aqui...

<https://www.figma.com/proto/UMW43Rm17DqLgLorqZqFTs/Sem-t%C3%ADtulo?node-id=1-4&t=WgXX6LJWkhuhL9nV-1>

**Professor:**

Leonardo Felipe Marinho Araujo

**Brasília – 2026.1**

## **Sumário**

|                                       |   |
|---------------------------------------|---|
| 1. INTRODUÇÃO .....                   | 1 |
| 2. JUSTIFICATIVA.....                 | 1 |
| 3. OBJETIVOS.....                     | 1 |
| 3.1. Objetivo Geral .....             | 1 |
| 3.2. Objetivos Específicos .....      | 1 |
| 4. DESCRIÇÃO DO SISTEMA PROPOSTO..... | 1 |

|        |                                  |   |
|--------|----------------------------------|---|
| 5.     | REQUISITOS.....                  | 2 |
| 5.1.   | Requisitos Funcionais.....       | 2 |
| 5.2.   | Requisitos Não Funcionais.....   | 2 |
| 5.3.   | Regras de Negócio .....          | 2 |
| 6.     | DIAGRAMAS UML .....              | 3 |
| 6.1.   | Diagrama de Casos de Uso .....   | 3 |
| 6.1.1. | Resumo .....                     | 3 |
| 6.1.2. | Desenho/imagem do diagrama ..... | 3 |
| 6.1.3. | Descrição dos Casos de Uso ..... | 3 |
| 6.2.   | Diagrama de Classe .....         | 4 |
| 6.2.1. | Resumo .....                     | 4 |
| 6.2.2. | Desenho/Imagem do Diagrama.....  | 4 |
| 6.3.   | Diagrama de Atividades .....     | 5 |
| 6.3.1. | Resumo .....                     | 5 |
| 6.3.2. | Desenho/Imagem do Diagrama.....  | 5 |
| 6.4.   | Diagrama de Sequência .....      | 5 |
| 6.4.1. | Resumo .....                     | 5 |
| 6.4.2. | Desenho/Imagem do Diagrama.....  | 6 |
| 7.     | PROTÓTIPOS .....                 | 6 |
| 8.     | CONCLUSÃO.....                   | 6 |
| 9.     | REFERÊNCIAS .....                | 6 |

## **1. INTRODUÇÃO**

Introdução: O trabalho consiste em buscar e resolver Falhas de conexão com Banco de Dados.

## **2. JUSTIFICATIVA**

Justificativa: O Projeto é importante por ser um erro que acontece frequentemente, que atinge tanto os desenvolvedores quanto os usuários comuns, do Banco de Dados. Nos dias atuais o problema persiste pelo fato da complexidade e da escala dos sistemas modernos, e qual a ideia? Criar uma resiliência no nível da aplicação, fazer com que ao invés do código dar um erro na tela do usuário, ele ser inteligente. Como? Simples realizando uma implementação do padrão Retry com Exponential Backoff, que nada mais é do que se caso a conexão falhar, o sistema tenta novamente de forma automática. Porém, para não bombardear o banco de dados que já pode estar sofrendo, ele espera um tempo progressivo (tenta em 1 segundo, depois em 2, depois em 4, depois em 8).

## **3. OBJETIVOS**

### **3.1. Objetivo Geral**

3.1 - O objetivo Geral do sistema basicamente é tentar ajudar no dia a dia de usuários, devs, dbas, analistas de suporte, e clientes de plataformas e aplicativos.

### **3.2. Objetivos Específicos**

- Requisitos funcionais - Detecção de falhas de conexão, tentativa automática com Exponential Backoff, configuração de parâmetros de Retry, registro de eventos em Log, exibição de mensagem amigável ao usuário, painel de monitoramento e relatório de disponibilidade.

- Requisitos não funcionais - Desempenho, disponibilidade, segurança, escalabilidade, usabilidade, manutenibilidade e compatibilidade.

#### **4. DESCRIÇÃO DO SISTEMA PROPOSTO**

O nexus se trata de um sistema de resiliência de conexão com banco de dados, que foi desenvolvido para atuar diretamente na camada de aplicação, afim de reduzir o impacto de falhas de conexão sobre os usuários finais. Ao invés de propagar erros imediatamente para a tela do usuário, o sistema aplicaria uma lógica inteligente para realizar uma recuperação automática, tornando o sistema mais tolerante a falhas transitórias, fazendo o máximo possível para que não chegue a precisar de uma intervenção humana.

#### **5. REQUISITOS**

##### **5.1. Requisitos Funcionais**

- Requisitos funcionais - Detecção de falhas de conexão, retentativa automática com Exponential Backoff, configuração de parâmetros de Retry, registro de eventos em Log, exibição de mensagem amigável ao usuário, painel de monitoramento e relatório de disponibilidade.

##### **5.2. Requisitos Não Funcionais**

- Requisitos não funcionais - Desempenho, disponibilidade, segurança, escalabilidade, usabilidade, manutenibilidade e compatibilidade.

### **5.3. Regras de Negócio**

RN01 – Conectada à RF02: o sistema deve realizar retentativas apenas para falhas que sejam classificadas como transitórias (ex.: timeout, conexão recusada temporariamente). Falhas permanentes, como credenciais inválidas, não devem disparar o mecanismo de repetição.

RN02 – Relacionada à RF02: o número máximo de tentativas padrão é 4, com intervalos de 1 segundo, 2 segundos, 4 segundos e 8 segundos. Esse número pode ser modificado pelo administrador, de acordo com a RF03, mas em nenhum caso deve ser abaixo de 1 ou acima de 10 tentativas.

RN03 – Complementar à RF03: quaisquer mudanças nos parâmetros de retry devem ser logadas em um registro de auditoria, indicando qual usuário efetuou a alteração e quando.

RN04 – Em conexão com a RF04: os logs devem ser mantidos por pelo menos 90 dias, e usuários que não possuem um perfil de administrador não devem ter a capacidade de excluí-los manualmente.

RN05 – Relacionada à RF05: a notificação de falha crítica deve ser emitida apenas após a conclusão de todas as tentativas configuradas, evitando alarmes falsos enquanto ainda se tenta reconectar.

RN06 – Complementar à RF06: a mensagem apresentada ao usuário final não deve incluir detalhes técnicos como stack trace, nome do host do banco ou código de erro interno.

RN07 – Em relação à RNF03: de forma alguma nenhuma string de conexão, senha ou token de autenticação deve ser gravado nos arquivos de log, mesmo que o aplicativo esteja em modo de depuração.

## **6. DIAGRAMAS UML**

### **6.1. Diagrama de Casos de Uso**

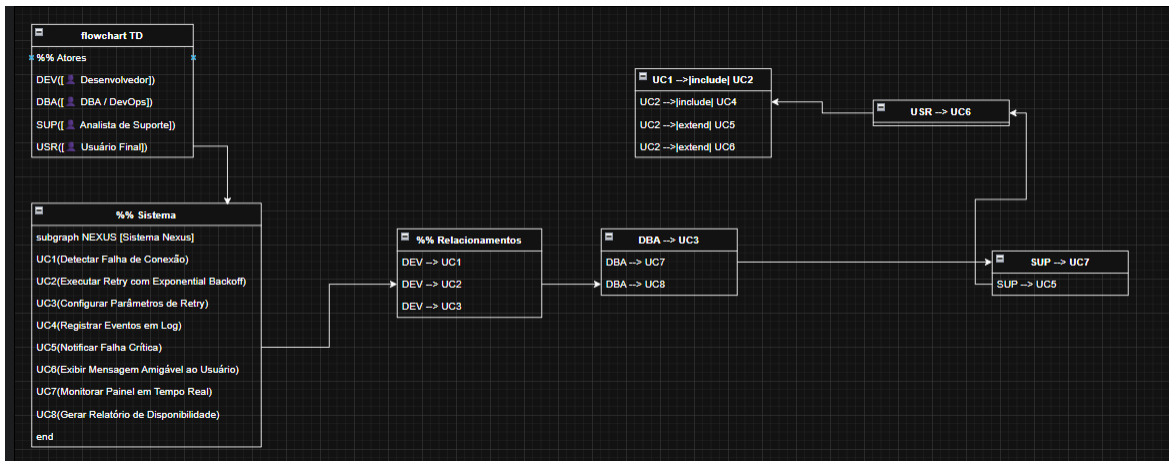
#### **6.1.1. Resumo**

O diagrama de casos de uso é um dos mais importantes da UML, visto que seu propósito é representar de forma visual e clara quais funcionalidades um sistema oferece, bem como quem são os usuários responsáveis por cada uma delas. Ele é utilizado principalmente na etapa de análise de requisitos, atuando como um intermediário entre os stakeholders, que frequentemente não têm um conhecimento técnico aprofundado, e a equipe de desenvolvimento, que precisa saber com precisão o que o sistema deve realizar.

Um diagrama de casos de uso é composto principalmente por três elementos: atores, que são representados com uma figura em forma de palito e representam tanto usuários humanos quanto sistemas externos e a aplicação; casos de uso — mostrados como elipses — representando o que um sistema faz ou o comportamento que será executado quando o sistema estiver em ação; e relacionamentos — que ligam um ator a um ou mais casos de uso; neste caso, ele pode especificar inclusão (>), extensão (> ) ou herança entre os atores.

O diagrama de casos de uso do sistema Nexus mostra a interação de cada tipo de usuário (Desenvolvedor, DBA, DevOps, Analista de Suporte e Usuário Final) com recursos como configuração do parâmetro de retentativa, monitoramento de eventos de falha, logs de consultas e recebimento de uma notificação para falhas críticas. Isso nos ajuda a identificar recursos que pertencem a Perfis específicos, versus todos os Perfis, assim auxiliando na definição de perfis de permissões e na validação de requisitos.

### 6.1.2. Desenho/imagem do diagrama



### 6.1.3. Descrição dos Casos de Uso

Uso: UC01 – Identificar Perda de Conexão

Ator(es): Programador

Descrição: O sistema é capaz de acompanhar as tentativas de conexão ao banco de dados e detectar automaticamente uma falha, classificando o tipo de erro que foi retornado.

Pré-requisito: A aplicação precisa estar em execução e deve haver uma tentativa de conexão com o banco de dados.

|                    |                                                                                                                                                            |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Caso de Uso:       | Executar Retry com Exponential Backoff.                                                                                                                    |
| Ator(es):          | Devs/DBAs/Usuários                                                                                                                                         |
| Descrição:         | Auxilio e assistência para a correção de erros.                                                                                                            |
| Pré-condição:      | A atribuição da i.a no sistema.                                                                                                                            |
| Fluxo Principal:   | Passo a passo da interação padrão                                                                                                                          |
| Fluxo Alternativo: | Se todas as tentativas forem esgotadas sem sucesso, o sistema dispara uma notificação de falha crítica (UC05) e exibe mensagem amigável ao usuário (UC06). |
| Pós-condições:     | A conexão é restabelecida com sucesso ou todas as tentativas vão se esgotar e as notificações e mensagens aos usuários são acionadas.                      |

Uso: UC02 – Executar Retry com Exponential Backoff.

**Ator(es):** Desenvolvedor.

**Descrição:** Após a detecção de uma falha, o sistema realiza novas tentativas de conexão automaticamente, com intervalos de espera progressivos.

**Pré-condição:** Uma falha de conexão deve ter sido detectada (UC01).

Ao ser identificada os parâmetros de retry devem estar configurados.

**Fluxo Principal:**

- O sistema registra a tentativa falha em log (UC04).
- O sistema aguarda o intervalo de espera progressivo. (1s, 2s, 4s ou 8s).
- O sistema realiza uma nova tentativa de conexão.
- Se a conexão for estabelecida, o fluxo da aplicação irá continuar normalmente.

**Fluxo Alternativo:**

- Se todas as tentativas forem sem sucesso, o sistema dispara uma notificação de falha crítica (UC05) e exibe mensagem amigável ao usuário (UC06).

## 6.2. Diagrama de Classe

### 6.2.1. Resumo

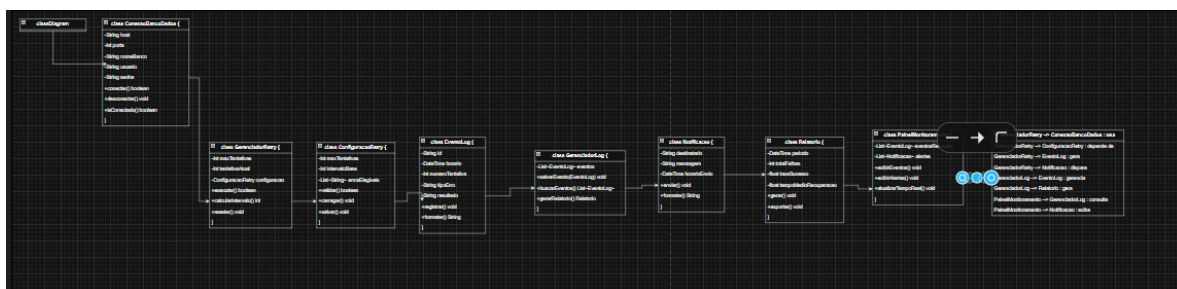
#### Resumo:

Os diagramas de classes são um dos diagramas estruturais mais utilizados na UML que permitem descrever a estrutura estática do sistema orientado ao objeto. Ele descreve as classes que formam o sistema juntamente com seus relacionamentos, atributos, métodos e podem servir como uma base para o desenvolvimento e organização do código fonte.

Os principais elementos da linguagem são: classes: são os retângulos com 3 depósitos (nome da classe, atributos da classe e operações da classe)



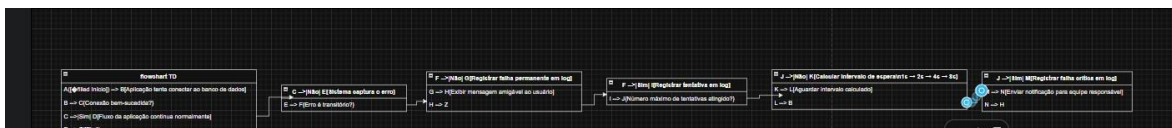
Dentro do ambiente do sistema Nexus, o diagrama de classe representa elementos chave associados no mecanismo de resiliência como a classe que modela as tentativas de conexão, eventos de log, parâmetros de nova tentativa e notificação como as classes. Este diagrama é muito importante para que o grupo de desenvolvimento entenda como as responsabilidades são compartilhadas entre os componentes do sistema e como eles interagem entre si.



complexos tanto do ponto de vista técnico quanto do ponto de vista das partes interessadas do negócio.

Seus elementos são: um nó inicial, representado por um círculo preenchido, que indica onde o fluxo começa; um nó final, representado por um círculo com borda dupla, que marca o fim de um fluxo em diagramas de atividades ou o fim do comportamento dentro de uma árvore de comportamento em diagramas de máquina de estados; atividades, representadas por retângulos com cantos arredondados, que representam as ações realizadas; nós de decisão, representados por losangos, que representam um ponto onde o fluxo pode se ramificar com base em uma condição; e bifurcações e junções, representadas por barras horizontais, que representam o paralelismo, onde o fluxo pode se dividir em múltiplos caminhos simultâneos ou fundir fluxos paralelos em um único caminho. No sistema Nexus, o diagrama de atividades também é muito apropriado para ilustrar todo o fluxo do mecanismo de Retentativa com Recuo Exponencial, desde a primeira tentativa de conexão, passando pelas decisões de classificação de erros, pelas novas tentativas com intervalos crescentes, até os possíveis resultados: reconectado, equipe notificada ou mensagem do usuário.

### 6.3.2. Desenho/Imagem do Diagrama



## 6.4. Diagrama de Sequência

### 6.4.1. Resumo

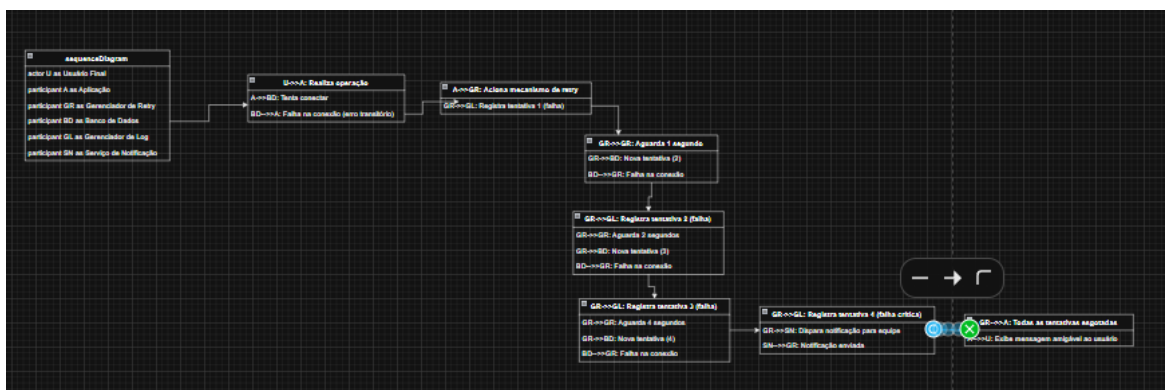
Resumo:

O diagrama de sequência é um tipo de diagrama comportamental da UML que apresenta um resultado esperado para a troca de mensagens entre os

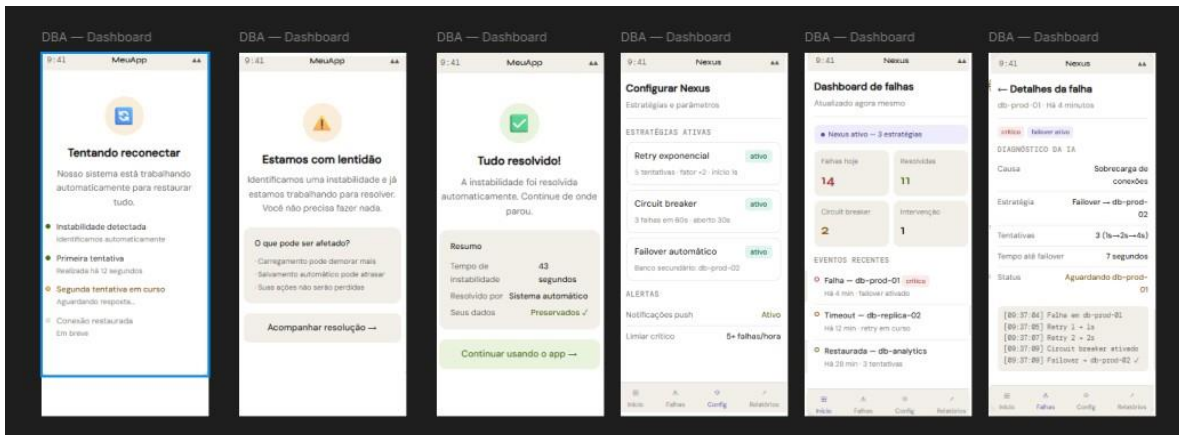
componentes de um sistema ao longo do tempo. Em contraste com o diagrama de atividade que enfatiza o fluxo de um processo, o diagrama de sequência tem como foco quem se comunica com quem e na ordem com que essas comunicações ocorrem, revelando a interação entre os elementos de um sistema em um contexto.

Seus principais componentes são: atores e objetos, localizados no topo do diagrama, que representam os participantes da interação; linhas de vida, linhas verticais tracejadas que ficam abaixo de cada participante e representam a existência do participante no tempo; mensagens, setas horizontais entre as linhas de vida que sinalizam chamadas, returns ou notificações; e blocos de ativação, retângulos que ficam sobre a linha de vida e denotam o quanto de tempo um participante está ativo realizando uma tarefa, processo ou operação. No contexto do sistema Nexus, o diagrama de sequência facilita a demonstração de uma tentativa malsucedida de conexão, interação entre o usuário final, a aplicação, o gerenciador de retry, o banco de dados, o gerenciador de log e o serviço de notificação e o processo de recuperação automática via Exponential Backoff.

#### 6.4.2. Desenho/Imagem do Diagrama



## 7. PROTÓTIPOS



Consiste basicamente em 6 telas, aonde 3 delas (as 3 da direita) são voltadas apenas para DEVs e DBAs, onde vão aparecer informações mais complexas sobre o sistema, como um dashboard de falhas que facilitará no reconhecimento dos erros e falhas do sistema, outra onde será os detalhes de cada falha e por último a página de configuração do nexus, onde será possível configurar o tempo e intensidade dos processos de retry. Ao lado esquerdo temos 3 telas, que são as que irão aparecer para os usuários finais, a tela de lentidão, que consiste na detecção de alguma falha por parte da I.A, a tela de tentando reconectar, que é quando a I.A está executando, e por último a tela de resolução.

## 8. CONCLUSÃO

A realização desse trabalho foi bastante produtiva para mim, pude aprender bastante coisa nova e principalmente surgir o desejo de levar esse projeto para frente. Escolhi esse tema justamente por conta da minha vontade de querer atuar na área de banco de dados, e pensar nessa resolução de um problema que é frequente foi algo bastante desafiador, aproveitei para estudar e aprender bastante coisa nessa área de inteligências artificiais e entender como elas funcionam, portanto foi um trabalho bastante desafiador e complexo, mas que me proporcionou grandes novas inovações e pensamentos.

## 9. REFERÊNCIAS

NYGARD, Michael T. **Release it!: design and deploy production-ready software**. 2. ed. Raleigh: Pragmatic Bookshelf, 2018.

MICROSOFT. **Retry pattern**. Microsoft Azure Architecture Center, 2023. Disponível em: <https://learn.microsoft.com/en-us/azure/architecture/patterns/retry>.

DRAW.IO. **Diagrams.net: ferramenta de modelagem UML**. Disponível em: <https://www.drawio.com>. Acesso em: maio 2026.

### Ficha de Autoavaliação do Grupo

- A ficha de autoavaliação deve ser única por grupo e inserida na última página do trabalho.
- O preenchimento deve ser feito em conjunto, de forma justa e responsável.
- O objetivo é registrar o nível de participação de cada integrante no desenvolvimento do projeto.
- Cada critério deve ser avaliado de 0 a 5, onde 0 significa ausência de contribuição e 5 significa contribuição máxima.

| Integrantes      | Participação nas discussões | Produção de conteúdo escrito | Contribuição nos diagramas UML | Colaboração na organização/formatação | Participação na apresentação |
|------------------|-----------------------------|------------------------------|--------------------------------|---------------------------------------|------------------------------|
| Nome completo 01 |                             |                              |                                |                                       |                              |
| Nome completo 02 |                             |                              |                                |                                       |                              |
| Nome completo 03 |                             |                              |                                |                                       |                              |
| Nome completo 04 |                             |                              |                                |                                       |                              |
| Nome completo 05 |                             |                              |                                |                                       |                              |